

Exercise 1-C (Summer 2026)

6.0 VU Advanced Database Systems

Information

Exercise Sheets

This exercise consists of **three exercise sheets** which will be available in TUWEL on:

- Exercise Sheet 1-A (PL/pgSQL): March 2nd
- Exercise Sheet 1-B (Datalog): March 4th
- Exercise Sheet 1-C (Common Table Expressions): March 6th

General

Work through **all three exercise sheets** and compile your answers in a **single report**. Submit the report and your source files as a **single ZIP (.zip)** file on TUWEL. The ZIP (.zip) file must contain:

- one PDF (.pdf) file,
- SQL (.sql) files containing your PostgreSQL commands (per exercise in sheet),
- Datalog (.dl) files containing your Datalog commands (per exercise in sheet).

SQL/Datalog file naming format: ex-⟨Exercise Sheet⟩-⟨Exercise Number⟩.⟨Question Number⟩.{sql/dl}

Submit the ZIP (.zip) file (max. 20 MB) and register for an exercise interview. **You can only receive points for this exercise if you attend the interview.** We expect you to explain your work in your report, i.e., it is not enough to only write the final answers. Show how you arrived at your answers and, where applicable, discuss your results. **We will not accept any handwritten reports!**

This exercise sheet involves working on a PostgreSQL DBMS and Datalog systems. For these exercises we provide you with web tools. For this particular exercise sheet you will use the PostgreSQL web tool, which is described in detail below.

Deadline

at latest **March 24th 9:00** Upload your submission on TUWEL
Do not forget! ⇒ Register for an exercise interview in TUWEL

Exercise Interviews

During the exercise interview, the correctness of your solution as well as your understanding of the underlying concepts will be assessed.

The scoring of your submission is primarily based on your performance at the exercise interview. Therefore it is possible (in extreme cases) to get 0 points even though the submitted solution was technically correct.

Please be punctual for your exercise interview. Otherwise we cannot guarantee that your full solution can be graded in your assigned time slot. Remember to have your student id with you for the exercise interview. It is not possible to score your solution without an id.

TUWEL Forum

You can use the course forum on TUWEL for clarifying questions regarding the exercise sheets. Please do not post your solutions (even partial) on the forum.

Postgres environment

For this exercise, we provide a web tool with a locally running PostgreSQL 16. You can access the tool at <https://dbai.tuwien.ac.at/adbs/>. On that page, click the *Open PostgreSQL Tool* button, to launch the PostgreSQL interface, as shown in Figure 1.

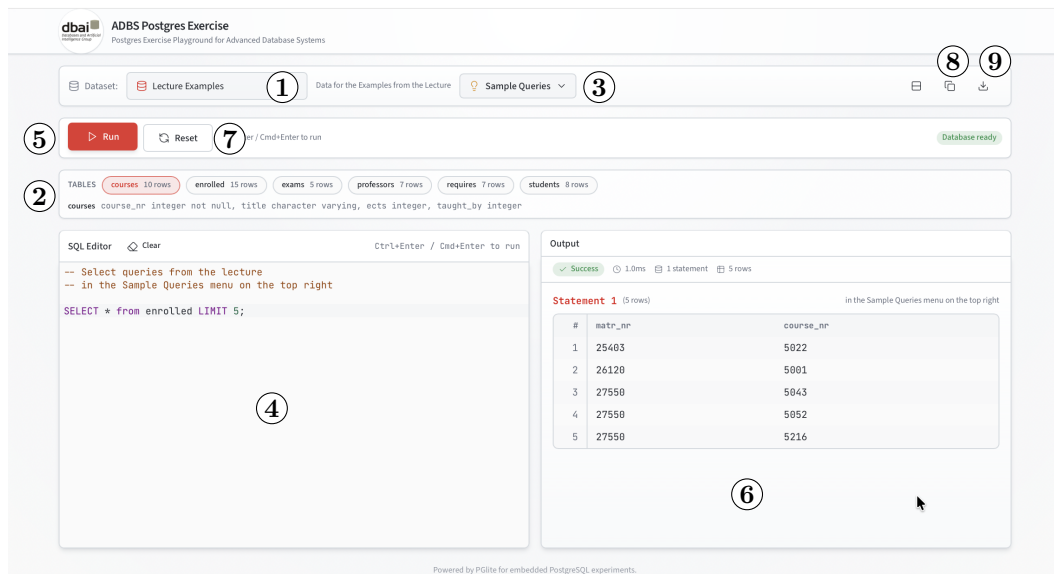


Figure 1: The PostgreSQL Tool

- ① You can use this drop-down menu to load one of the listed datasets.
- ② Once a dataset is loaded, the tables in the dataset and their details will appear in the information bar. Selecting a table will display its schema details.
- ③ In some cases, sample queries are provided for you to explore. You can choose an existing sample query from this drop-down menu and it will appear in ④.
- ④ You can type new queries or modify the sample queries using this editor.
- ⑤ Click this Run button to execute your queries in your editor ④.
- ⑥ You can see the outputs of your executed queries here.
- ⑦ If you wish to start over, this Reset button will clear all changes in the tool and datasets. You can reload a dataset from the drop-down menu ① and start over.
- ⑧ This button will allow you to copy your queries in the editor.
- ⑨ This button will allow you to download your queries in the editor into a SQL file.

Exercises

In Exercises 5 and 6, you will explore the concept of common table expressions (CTEs) to gain hands-on experience and understand their benefits in a DBMS. The lecture on CTEs offered a comprehensive overview of their usage and related SQL features, which provides all the necessary background to

complete these exercises. For additional details or examples, we recommend the official PostgreSQL documentation Chapter ¹.

These exercises also use the "Great Outdoors" (GO) Sales dataset from IBM, which contains information on daily sales, sales methods, retailers, and products for a fictitious outdoor equipment retail chain. (Available here : <https://relational.fel.cvut.cz/dataset/GOSales>). Use the PostgreSQL web tool and switch to the "GO Sales" dataset to test and answer the following scenarios.

Given in Figure 2 is the schema of the dataset.

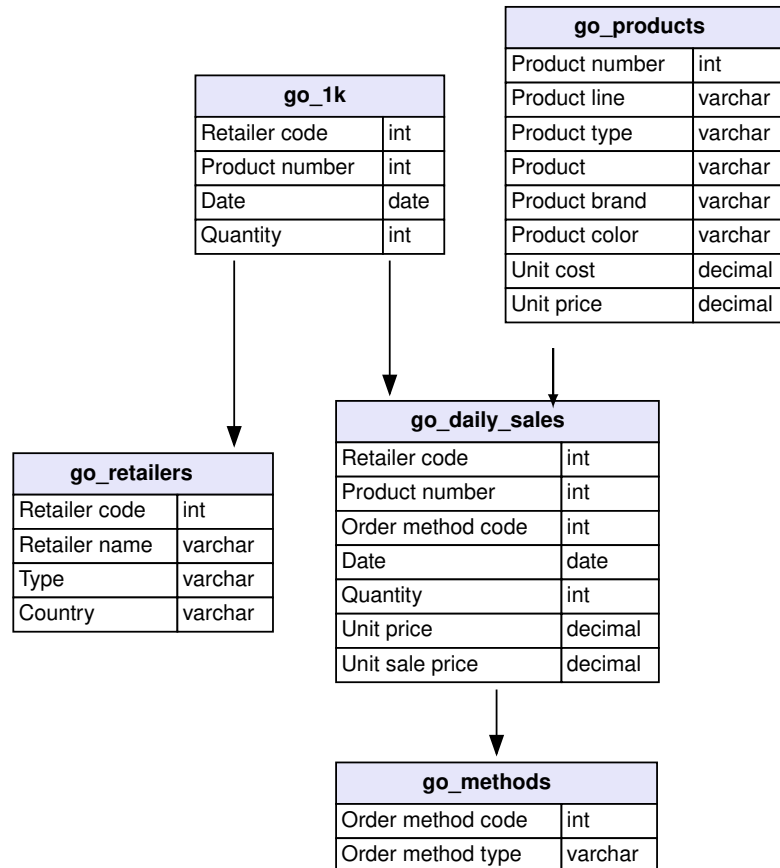


Figure 2: The database schema

Exercise 5 (CTEs) [2 points] The GO team's dashboard queries monthly sales totals for each product more frequently than initially expected. Currently, these totals are computed on the fly from the `go_daily_sales` table using a stored function (Recall Exercise 1), which has become inefficient due to the increased query frequency. To improve performance, the team decides to use a materialized view that stores precomputed monthly aggregates, allowing the dashboard to retrieve results quickly and efficiently.

1. Create a materialized view `monthly_product_sales` that aggregates sales data per product, per month, and per year. *No need to include months without any sales for a product.*
2. Write a CTE to identify the top selling month per product within 2018, using the above materialized view.

Hint: You can use the `RANK()` function over a window defined with `PARTITION BY` function to rank months for each product based on total revenue.

¹<https://www.postgresql.org/docs/>

Exercise 6 (CTEs with Recursion) [3 points] The GO team wants to display the longest consecutive daily sales streak for each product in their dashboard.

1. Using the sales data (`go_daily_sales`), create a recursive CTE that returns, for each product, the maximum number of consecutive days with at least one sale. The output should include `product_number` and `longest_streak`, ordered by streak length in descending order.

The recursive CTE should:

- identify the unique sales dates per product,
- find consecutive sequences using these unique dates,
- compute the maximum consecutive sequence per product and order the output rows by their length

Hint: You can use `INTERVAL '1 day'` to increment the dates.

2. Modify the previous solution by creating a temporary table to store the unique sales dates per product, and then use this table in the recursive CTE instead of generating the distinct dates inline.